Advanced Encryption Standard (AES) Engine v1.1

LogiCORE IP Product Guide

Vivado Design Suite

PG383 (v1.1) April 26, 2022

Xilinx is creating an environment where employees, customers, and partners feel welcome and included. To that end, we’re removing non-inclusive language from our products and related collateral. We’ve launched an internal initiative to remove language that could exclude people or reinforce historical biases, including terms embedded in our software and IPs. You may still find examples of non-inclusive language in our older products as we work to make these changes and align with evolving industry standards. Follow this link for more information.

Table of Contents

Chapter 1: Introduction 4
Features 4
IP Facts 5

Chapter 2: Overview 6
Core Overview 6
Applications 7
Unsupported Features 7
Licensing and Ordering 7

Chapter 3: Product Specification 9
Standards 10
Performance 11
Resource Use 13
Port Descriptions 13

Chapter 4: Designing with the Core 15
General Design Guidelines 15
Clocking 16
Resets 16

Chapter 5: Design Flow Steps 17
Customizing and Generating the Core 17
Constraining the Core 19
Simulation 20
Synthesis and Implementation 20

Chapter 6: Example Design 21

Appendix A: Verification, Compliance, and Interoperability 23

Appendix B: Upgrading 24

Appendix C: Debugging **25**
Finding Help on Xilinx.com 25
Debug Tools 26
Simulation Debug 26
Interface Debug 27

Appendix D: Additional Resources and Legal Notices **28**
Xilinx Resources 28
Documentation Navigator and Design Hubs 28
References 28
Revision History 29
Please Read: Important Legal Notices 30

Chapter 1

Introduction

The LogiCORE™ IP Advanced Encryption Standard (AES) core provides encryption and decryption functions compliant to the Federal Information Processing Standard (FIPS) Publication 197 from the U.S. National Institute of Standards and Technology (NIST) and other standards, if applicable.

The core comes with a standard AXI4-Stream interface for data and a convenient AXI4-Stream-like interface for key and other metadata. This enables you to integrate the core seamlessly into your existing systems without needing any additional components/IPs, thus reducing the time to advance the design significantly.

Features

- Compliant to NIST FIPS PUB 197
- AXI4-Stream interface with 128, 256, or 512 data-width options
- Built-in key expansion for both encryption and decryption
- Ultra-high speeds up to 16 GB/s at 250 MHz
- Built-in configurable tweak calculation engine for XTS mode of operation
- Both pipelined and lite design variants to choose between resource and performance
- Plug and play without external FIFOs
- Key and initialization vector (IV) prefetch to achieve zero latency between successive data units

IP Facts

LogiCORE™ IP Facts Table	
Core Specifics	
Supported Device Family[1]	UltraScale+™ families and UltraScale™ families
Supported User Interfaces	AXI4-Stream
Resources	Performance and Resource Use web page
Provided with Core	
Design Files	Encrypted RTL
Example Design	Verilog
Test Bench	Verilog
Constraints File	Xilinx Constraints File
Simulation Model	Not Provided
Supported S/W Driver	N/A
Tested Design Flows[2]	
Design Entry	Vivado® Design Suite
Simulation	For supported simulators, see the Xilinx Design Tools: Release Notes Guide.
Synthesis	Vivado Synthesis
Support	
Release Notes and Known Issues	Master Answer Record: N/A
All Vivado IP Change Logs	Master Vivado IP Change Logs: 72775
Xilinx Support web page	

Notes:

1. For a complete list of supported devices, see the Vivado IP catalog.
2. For the supported versions of third-party tools, see the Xilinx Design Tools: Release Notes Guide.

Chapter 2

Overview

Core Overview

Data center storage is evolving at a rapid pace due to the emergence of newer storage technologies such as NVMe and 3D-XPoint coupled with the introduction of SmartNICs that can offload some of the critical processing to the network interface controller while being programmable. This puts a demand on system components to support line-rate processing of data. Encryption is at the heart of any such application and the Xilinx® AES IP has been designed to meet the demand for high performance encryption cores that can be seamlessly integrated into these systems.

These cores can also be used for applications ranging from communication systems to advanced driver-assistance systems (ADAS) and self-driving cars. The core also supports a lightweight, low resource count variant for applications such as smart card readers that do not need to perform at high-speeds. The core provides industry-leading performance enabled by several patent-pending architectural innovations.

The major components of the core are the AES encryption/decryption engines, the metadata generation block, key expansion, and control logic. The AES engines are swapped between their high and low-throughput variants based on the option you selected.

The core supports the following AES variants:

- AES-ECB-256
- AES-ECB-192
- AES-ECB-128
- AES-CFB128-256
- AES-CFB128-192
- AES-CFB128-128
- AES-XTS-256 (without CTS)
- AES-XTS-128 (without CTS)

Applications

The AES core has several configuration options to meet the requirements of a wide range of applications from high-speed data center storage and SmartNICs to smart cards and electronic financial transactions.

Unsupported Features

The following feature of the standard is not supported in the core:

- XTS mode of operation does not support ciphertext stealing (CTS)

Licensing and Ordering

This Xilinx® LogiCORE™ IP module is provided under the terms of the Xilinx Core License Agreement. The module is shipped as part of the Vivado® Design Suite. For full access to all core functionalities in simulation and in hardware, you must purchase a license for the core. To generate a full license, visit the product licensing web page. Evaluation licenses and hardware timeout licenses might be available for this core. Contact your local Xilinx sales representative for information about pricing and availability.

***Note*:** To verify that you need a license, check the License column of the IP Catalog. Included means that a license is included with the Vivado® Design Suite; Purchase means that you have to purchase a license to use the core.

For more information about this core, visit the AES product web page.

Information about other Xilinx® LogiCORE™ IP modules is available at the Xilinx Intellectual Property page. For information about pricing and availability of other Xilinx LogiCORE IP modules and tools, contact your local Xilinx sales representative.

License Checkers

If the IP requires a license key, the key must be verified. The Vivado® design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation

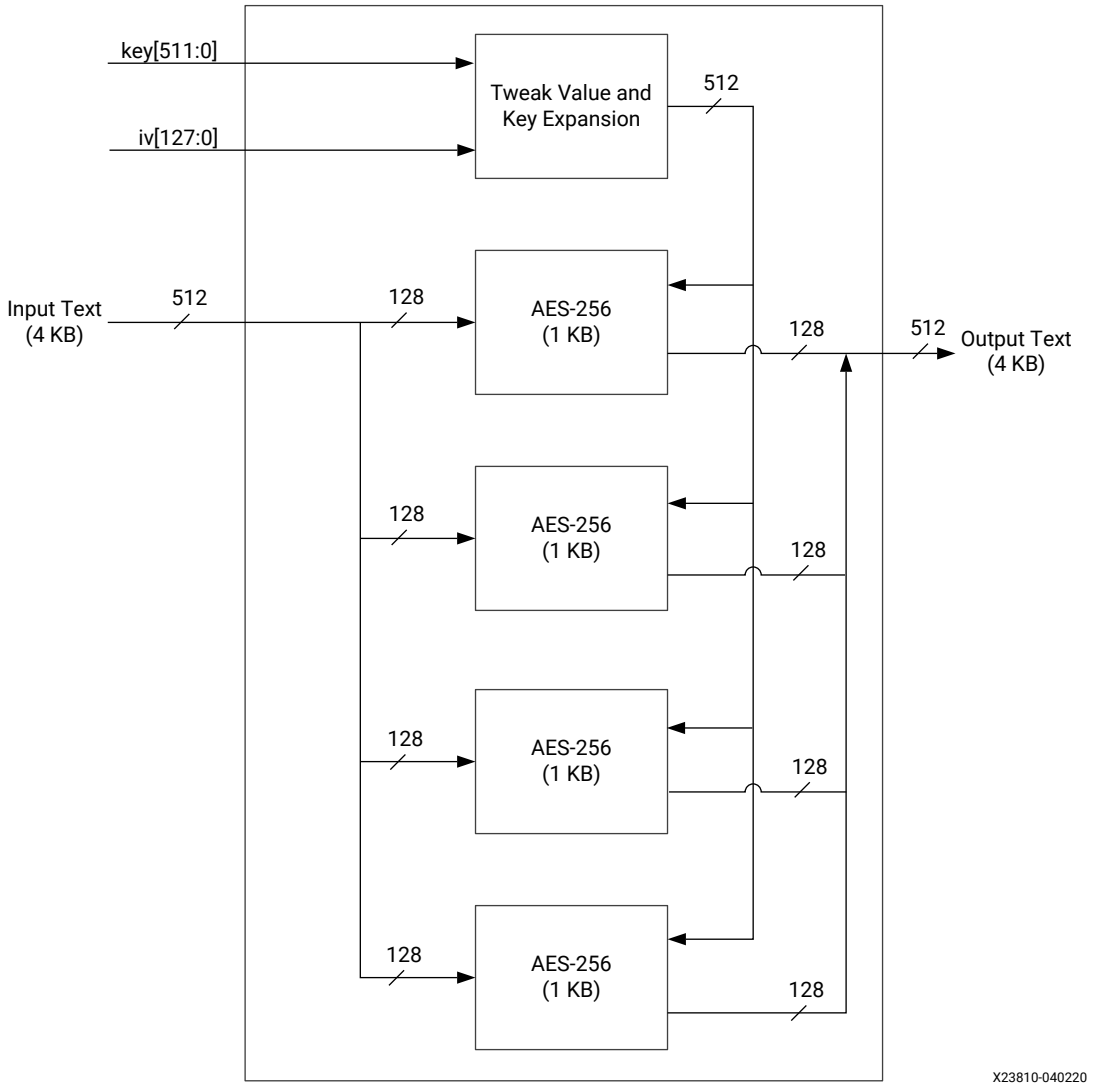
- write_bitstream (Tcl command)

IMPORTANT! *IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.*

Product Specification

The functional block diagram of the core is shown in the following figure.

Figure 1: **Core Block Diagram**



The core instantiates several AES engines internally based on the AXI4-Stream data width selected by you. It consists of a scalable tweak-calculation that generates several tweak values in a single clock-cycle so that all the engines can process data simultaneously without getting stalled. The key expansion module generates the round keys independently and provides them to all the engines.

When high-throughput mode is selected, the engines use a pipelined design that enables the core to provide data every cycle at the output. The high-throughput core also supports prefetch which enables you to provide the key and IV for the next packet while the current packet is being driven to the core. This enables the core to mask the latency that is introduced due to the generation of tweak or other metadata that must by ready before the engine can start accepting data. Thus, the design allows you to send in packets back-to-back without having to wait for the metadata to be ready for successive packets. This is useful in block-storage applications where the keys for all the blocks are pre-generated and stored in some memory within the system.

Prefetch is achieved by having a handshake mechanism for the key and IV. You are expected to drive the `key_valid` signal while sending valid key and IV. The core asserts the `key_fetch` signal to indicate that it is ready to take the next set of key and IV. The key and IV are latched only when both the `key_valid` and `key_fetch` are asserted during a particular clock cycle. The controller within the core ensures that the keys are managed properly so that they do not get overwritten when you feed the key and IV for the next packet. It also ensures that the keys are provided to the pipeline based on the state of movement of data through it.

Standards

This core adheres to the following standards:

- AES Engine: Federal Information Processing Standards Publication 197 - ADVANCED ENCRYPTION STANDARD (AES) (https://www.nist.gov/publications/advanced-encryption-standard-aes)
- AES ECB, CFB128: NIST Special Publication 800-38A - Recommendation for Block Cipher Modes of Operation (https://csrc.nist.gov/publications/detail/sp/800-38a/final)
- AES XTS:
  - NIST Special Publication 800-38E - Recommendation for Block Cipher Modes of Operation: The XTS-AES Mode for Confidentiality on Storage Devices (https://csrc.nist.gov/publications/detail/sp/800-38e/final)
  - IEEE P1619™ Standard for Cryptographic Protection of Data on Block-Oriented Storage Devices (https://ieeexplore.ieee.org/document/8637988)

Performance

For full details about performance and resource use, visit the Performance and Resource Use web page.

Maximum Frequencies

The core has been tested for all configurations at 250 MHz.

Latency

The latency information consists of two components: initial core readiness latency and the engine input-to-output latency. Depending on the variant of the AES algorithm being used, the readiness latency numbers vary. The number of cycles taken by the engine differ by one cycle between the high-throughput pipelined and the low-throughput variant. The core is designed to mask the readiness latency between successive data units by prefetching the key, IV, and other metadata in advance while processing the current packet. Thus, if the next unit's keys are provided within a certain number of cycles of the core requesting them, the design can achieve zero cycle latency between successive units.

Table 1: **Latency Table**

Encryption/ Decryption	Throughput Mode	Key Size	AES Mode	Key Expansion Latency[1]	Input to Output Latency[2]	Total Latency
Encryption	High-Throughput Mode	256	XTS	17	15	32
			ECB	1	15	16
		192	ECB	1	13	14
		128	XTS	13	11	24
			ECB	1	11	12
	Low-Throughput Mode	256	XTS	17	16	33
			ECB	1	16	17
			CFB128	1	16	17
		192	ECB	1	14	15
			CFB128	1	14	15
		128	XTS	13	12	25
			ECB	1	12	13
			CFB128	1	12	13

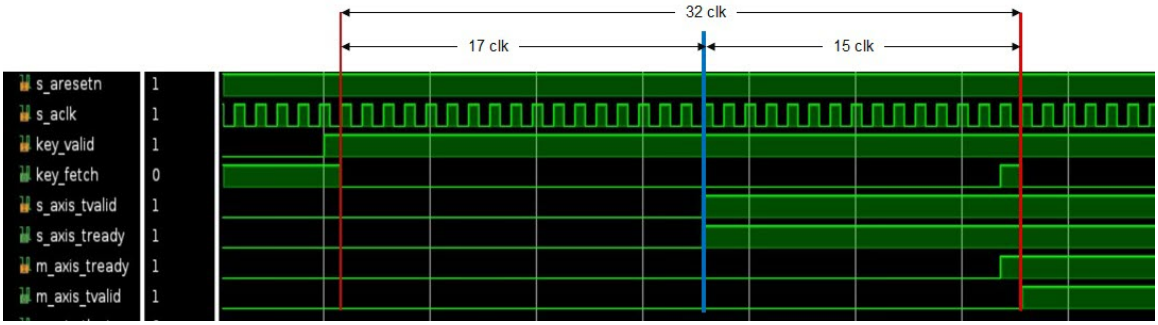
Table 1: **Latency Table** *(cont'd)*

Encryption/ Decryption	Throughput Mode	Key Size	AES Mode	Key Expansion Latency[1]	Input to Output Latency[2]	Total Latency
Decryption	High-Throughput Mode	256	XTS	31	15	46
			ECB	15	15	30
			CFB128	1	15	16
		192	ECB	13	13	26
			CFB128	1	13	14
		128	XTS	23	11	34
			ECB	11	11	22
			CFB128	1	11	12
	Low-Throughput Mode	256	XTS	31	16	47
			ECB	15	16	31
			CFB128	1	16	17
		192	ECB	13	14	27
			CFB128	1	14	15
		128	XTS	23	12	35
			ECB	11	12	23
			CFB128	1	12	13

1. From key acceptance to input data acceptance
2. From input data acceptance to output data

***Note*:** Input data is plain text for encryption and cipher text for decryption. Output data is cipher text for encryption and plain text for decryption.

Figure 2: **Latency Example**



The simulation waveform shows encryption latency for XTS mode with key size width of 256-bit.

Throughput

The high-throughput variant can achieve up to 128 Gb/s of throughput across packets for a 512-bit interface for packets of size 4 KB or greater. For smaller packets, performance is reduced based on their size. For applications that do not demand such performance, the low-throughput variant can be used which can run up to speeds of ~900 MB/s for a 512-bit interface. The throughput scales down based on the data width for both the variants.

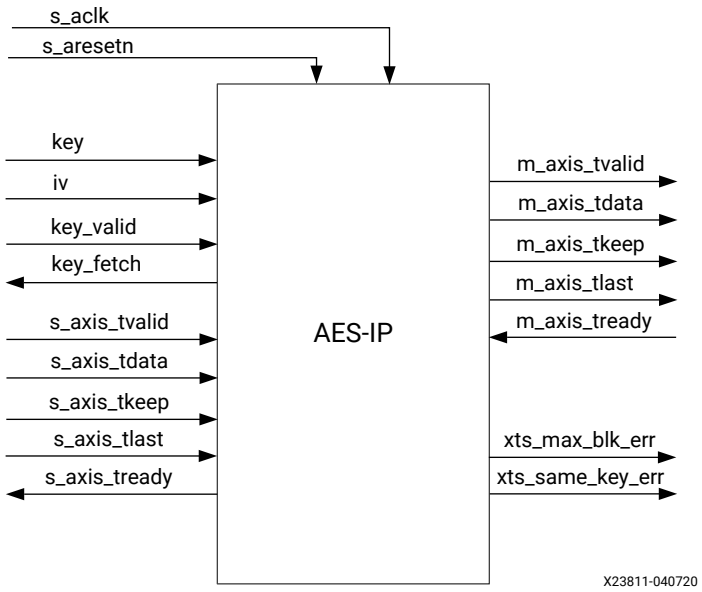
Resource Use

For full details about performance and resource use, visit the Performance and Resource Use web page.

Port Descriptions

The core interfaces are shown in the following figure.

Figure 3: **Core Ports**



AES Ports

Table 2: **AES Ports**

Port Name	I/O	Clock	Description
key_valid	I	s_aclk	Key valid pulse signal to indicate that metadata (key/IV) is valid.
key	I	s_aclk	Key for encryption/decryption, must be valid when key_valid is asserted.
iv	I	s_aclk	Initialization vector (IV) for encryption/decryption, must be valid when key_valid is asserted.
key_fetch	O	s_aclk	Indication from core that it is ready to accept metadata (key/IV). Metadata is latched when both key_valid and key_fetch are asserted simultaneously.
s_aclk	I		Clock signal for the core.
s_aresetn	I	s_aclk	Active-Low reset signal for the core.
s_axis_tvalid	I	s_aclk	Input AXI4-Stream data valid signal.
s_axis_tdata	I	s_aclk	Input AXI4-Stream data.
s_axis_tkeep	I	s_aclk	Input AXI4-Stream signal to indicate which bytes in the data are valid.
s_axis_tlast	I	s_aclk	Input AXI4-Stream signal to indicate last beat of the current packet.
s_axis_tready	O	s_aclk	Output AXI4-Stream signal to indicate that the core is ready to consume another beat of data.
m_axis_tvalid	O	s_aclk	Output AXI4-Stream data valid signal.
m_axis_tdata	O	s_aclk	Output AXI4-Stream data.
m_axis_tkeep	O	s_aclk	Output AXI4-Stream signal to indicate which bytes in the data are valid.
m_axis_tlast	O	s_aclk	Output AXI4-Stream signal to indicate last beat of the current packet.
m_axis_tready	I	s_aclk	Input AXI4-Stream signal to indicate that the core is ready to consume another beat of data.
xts_max_blk_err	O	s_aclk	XTS Mode Error Indication when number of blocks/packets in a data unit crosses 2^{20}.
xts_same_key_err	O	s_aclk	XTS Mode Error Indication when Key-1 and Key-2 within the same key are the same.

Chapter 4

Designing with the Core

This section includes guidelines and additional information to facilitate designing with the core.

General Design Guidelines

Use the Example Design

Each instance of the AES core created by the Vivado design tool is delivered with an example design that can be implemented in a device and then simulated. This design can be used as a starting point for your own design or can be used to sanity-check your application in the event of difficulty. See the Example Design content for information about using and customizing the example designs for the core.

Registering Signals

To simplify timing and increase system performance in a programmable device design, keep all inputs and outputs registered between the user application and the core. This means that all inputs and outputs from the user application should come from, or connect to, a flip-flop. While registering signals might not be possible for all paths, it simplifies timing analysis and makes it easier for the Xilinx® tools to place and route the design.

Recognize Timing Critical Signals

The constraints provided with the example design identify the critical signals and timing constraints that should be applied.

Make Only Allowed Modifications

You should not modify the core. Any modifications can have adverse effects on system timing and protocol compliance. Supported user configurations of the core can only be made by selecting the options in the customization IP dialog box when the core is generated.

Clocking

This core runs on a single clock through the port named `s_aclk`. The core has been tested to meet timing at 250 MHz when measured at the IP level. Meeting the same frequency when the core is inserted into a system is subject to system-design, device congestion, and usage of timing best practices among other factors.

Resets

This core includes an active-Low reset signal. There are no specific requirements with respect to the assertion of reset to the core.

Chapter 5

Design Flow Steps

This section describes customizing and generating the core, constraining the core, and the simulation, synthesis, and implementation steps that are specific to this IP core. More detailed information about the standard Vivado® design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994)
- *Vivado Design Suite User Guide: Designing with IP* (UG896)
- *Vivado Design Suite User Guide: Getting Started* (UG910)
- *Vivado Design Suite User Guide: Logic Simulation* (UG900)

Customizing and Generating the Core

This section includes information about using Xilinx® tools to customize and generate the core in the Vivado® Design Suite.

If you are customizing and generating the core in the Vivado IP integrator, see the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994) for detailed information. IP integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, run the `validate_bd_design` command in the Tcl console.

You can customize the IP for use in your design by specifying values for the various parameters associated with the IP core using the following steps:

1. Select the IP from the IP catalog.
2. Double-click the selected IP or select the Customize IP command from the toolbar or right-click menu.

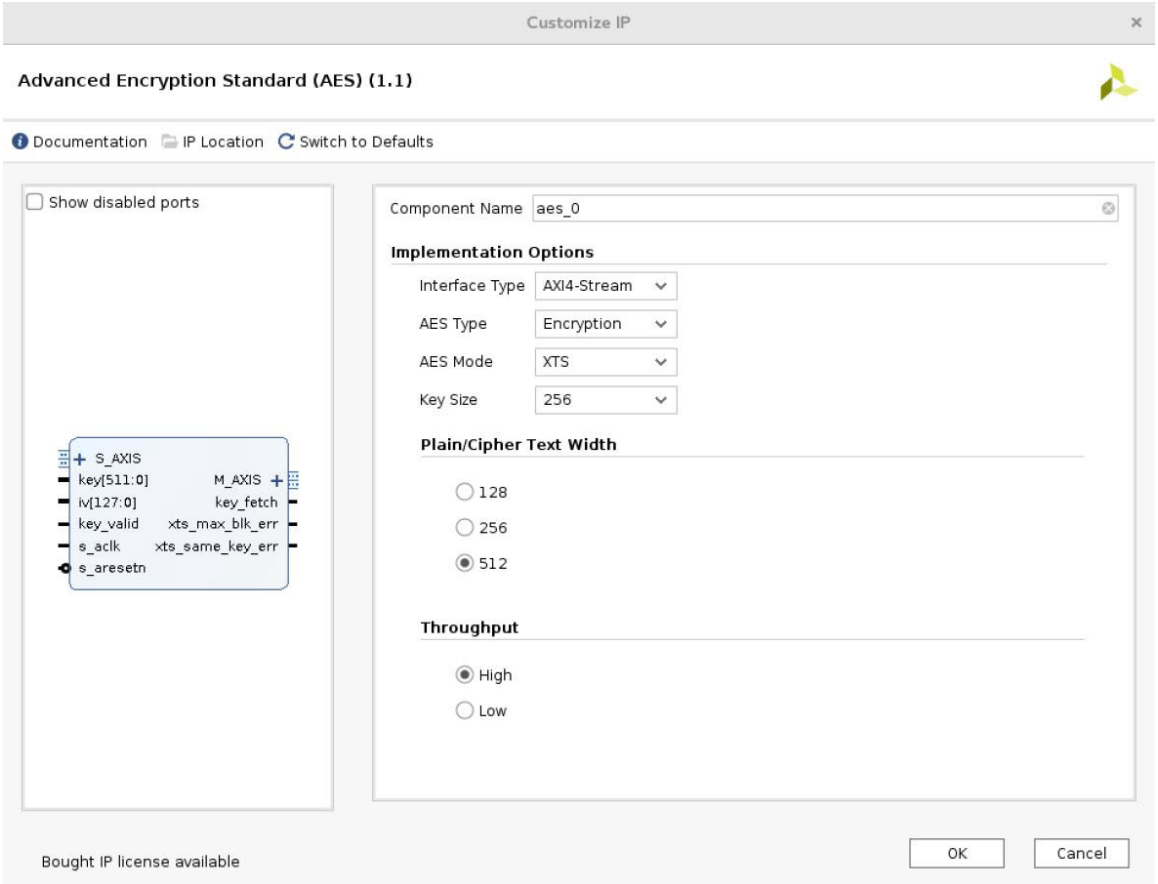
For details, see the *Vivado Design Suite User Guide: Designing with IP* (UG896) and the *Vivado Design Suite User Guide: Getting Started* (UG910).

Figures in this chapter are illustrations of the Vivado IDE. The layout depicted here might vary from the current version.

AES Parameter Tab

The AES parameter tab is shown below.

Figure 4: **AES Parameter Tab**



The options for the AES core are:

- **Interface Type:** Currently, only AXI4-Stream interface is supported.
- **AES Type:** Choose between Encryption or Decryption.
- **AES Mode:** Select between the supported modes of operation. Currently supported modes are ECB, CFB128, and XTS (without CTS).

  Note: The drop-down menu will only list modes for valid licenses that have been purchased. While generating the core, the Vivado IDE would flag Critical Warnings for all modes that licenses are not available, these can be ignored.

- **Key Size:** Choose the width of the key. Options are subject to the mode of operation. For example, ECB mode supports 128, 192, and 256-bit keys while XTS mode only supports 128-bit and 256-bit key variants.

- **Plain/Cipher Text Width:** Select the width of the AXI4-Stream data path; applies to both input and output interface.
- **Throughput:** Choose between High and Low throughput variants which trade off between resource utilization and performance.

  ***Note*:** In high-throughput mode, CFB128 mode supports only decryption.

User Parameters

The following table shows the relationship between the fields in the Vivado® IDE and the user parameters (which can be viewed in the Tcl Console).

Table 3: **User Parameters**

Vivado IDE Parameter/Value[1]	User Parameter/Value	Default Value
Interface Type	INTERFACE_TYPE	AXI4-Stream
AES Type	AES_TYPE	Encryption
AES Mode	AES_MODE	XTS
Key Size	KEY_SIZE	256
Plain/Cipher Text Width	DIN_WIDTH	512
Throughput	THROUGHPUT	High

Notes:

1. Parameter values are listed in the table where the Vivado IDE parameter value differs from the user parameter value. Such values are shown in this table as indented below the associated parameter.

Output Generation

For details, see the *Vivado Design Suite User Guide: Designing with IP* (UG896).

Constraining the Core

Required Constraints

This section is not applicable for this IP core.

Device, Package, and Speed Grade Selections

This section is not applicable for this IP core.

Clock Frequencies

This section is not applicable for this IP core.

Clock Management

This section is not applicable for this IP core.

Clock Placement

This section is not applicable for this IP core.

Banking

This section is not applicable for this IP core.

Transceiver Placement

This section is not applicable for this IP core.

I/O Standard and Placement

This section is not applicable for this IP core.

Simulation

For comprehensive information about Vivado® simulation components, as well as information about using supported third-party tools, see the *Vivado Design Suite User Guide: Logic Simulation* (UG900).

Synthesis and Implementation

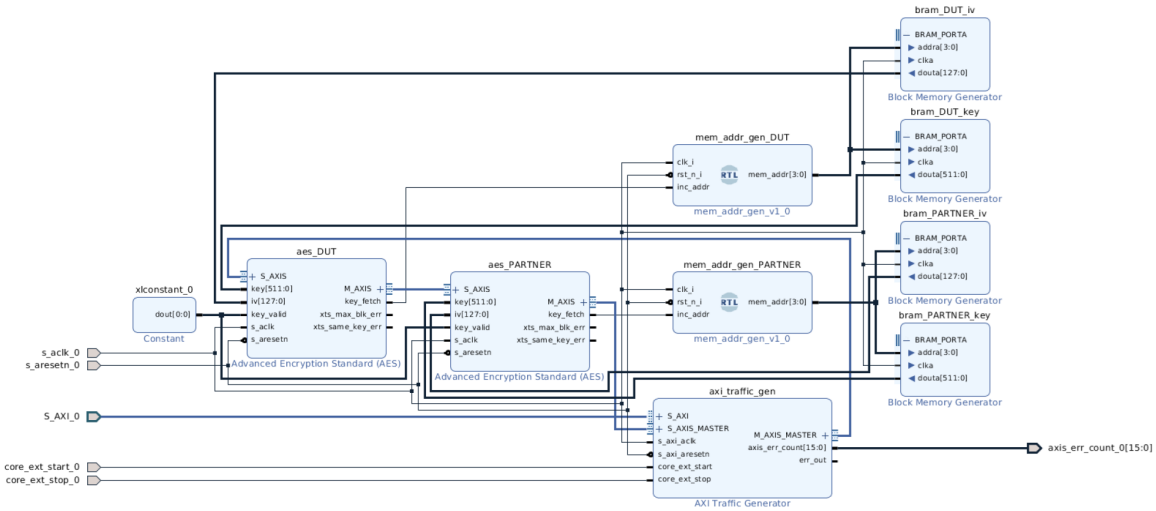
For details about synthesis and implementation, see the *Vivado Design Suite User Guide: Designing with IP* (UG896).

Example Design

This chapter provides information about the example design, including a description of the files and the directory structure generated by the Xilinx® Vivado® Design Suite, the purpose and contents of the provided scripts, the contents of the example HDL wrappers, and the operation of the demonstration test bench.

The following figure shows a snapshot of the example design for a core generated with the default configuration.

Figure 5: **Core Example Design**



The example design has a self-checking setup in which random AXI4-Stream traffic is generated by the AXI Traffic Generator IP, and sent to the DUT to encrypt/decrypt the data. The output of the DUT is sent to a DUT partner which is configured as the opposite type of the DUT. This means that a DUT generated for encryption would be paired with a partner that would run decryption and vice versa, keeping all the other parameters the same. The output of the partner is sent back to the AXI Traffic Generator to compare with the original data. A test bench is provided to run the example design and then print the result of the comparison.

The example design also instantiate block RAMs through the block Memory Generator IP for key and IV for all the blocks in the data for both the DUT and the partner. It also includes the `mem_addr_gen_v1_0` module that uses the `key_fetch` indication from the IP to increment the block RAM address and fetch the next set of metadata from the respective block RAMs. The `key_valid` input to the core is tied to logic "1" because all the keys and IV are readily available in the block RAM. This subsystem design is inspired from a typical storage encryption solution commonly used in the data centers.

***Note*:** When the DUT is generated for the CFB128 Mode Decryption, the example design would force the DUT and the DUT partner to run in a 128-bit Low throughput mode because the CFB128 Mode Encryption does not support any other configuration. As a result, a suitable DUT partner cannot be generated for any other configuration in this case.

Appendix A

Verification, Compliance, and Interoperability

Verification

The AES IP core has been verified in simulation. A highly parameterizable constrained random simulation test suite has been used to verify the core. Tests included:

- Random configuration of parameters
- Data integrity check and comparison against results from standard C-based libraries
- Testing against standard NIST vectors for respective variants
- Performance and latency checks
- Reporting of XTS errors

Compliance Testing

Version 1.0 of the core has passed NIST Cryptographic Algorithm Validation Program (CAVP) certification which provides validation testing of approved (that is, FIPS-approved and NIST-recommended) cryptographic algorithms and their individual components for all the supported configurations. More information related to the certification can be found in the following links hosted on the NIST COMPUTER SECURITY RESOURCE CENTER website:

- https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/details?product=12164
- https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/details?product=12163

Version 1.1 currently available in the Vivado® IP catalog includes some infrastructural changes to the core when compared to version 1.0. While Xilinx® believes version 1.1 is still compliant to the CAVP certification because no changes have been made to the algorithmic implementation of the core, customers requiring re-certification of the IP can contact Xilinx through standard support channels to seek further guidance.

Appendix B

Upgrading

This appendix is not applicable for the first release of the core.

Appendix C

Debugging

This appendix includes details about resources available on the Xilinx® Support website and debugging tools.

If the IP requires a license key, the key must be verified. The Vivado® design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation
- write_bitstream (Tcl command)

IMPORTANT! *IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.*

Finding Help on Xilinx.com

To help in the design and debug process when using the core, the Xilinx Support web page contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support. The Xilinx Community Forums are also available where members can learn, participate, share, and ask questions about Xilinx solutions.

Documentation

This product guide is the main document associated with the core. This guide, along with documentation related to all products that aid in the design process, can be found on the Xilinx Support web page or by using the Xilinx® Documentation Navigator. Download the Xilinx Documentation Navigator from the Downloads page. For more information about this tool and the features available, open the online help after installation.

Technical Support

Xilinx provides technical support on the Xilinx Community Forums for this LogiCORE™ IP product when used as described in the product documentation. Xilinx cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To ask questions, navigate to the Xilinx Community Forums.

Debug Tools

There are many tools available to address AES design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The Vivado® Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in Xilinx® devices.

The Vivado logic analyzer is used to interact with the logic debug LogiCORE IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the *Vivado Design Suite User Guide: Programming and Debugging* (UG908).

Simulation Debug

In XTS mode of operation, two error indications are provided by the core as per the specifications and certification requirements through separate output ports. Although the `xts_max_blk_err` condition would deassert the `s_axis_tready` and prohibit any more packets from entering the core, the `xts_same_key_err` indication is used to alert you and does not stop the functioning of the core. This method was done because the size of a data unit

not being more than 2^{20} blocks is a mandatory requirement for any implementation of the XTS algorithm, the requirement for the two keys within an XTS key not being the same is a security recommendation and not mandatory from the AES IP perspective. The core still provides you this indication to find any flaws in system design and thus the usage of the `xts_same_key_err` port is left to you.

If `xts_max_blk_err` is asserted, you must apply a reset before the core can start functioning again. Note that the output for all the data that has already been sent to the core can be expected at the output interface.

For more information on these errors, see the following links:

- *The XTS-AES Validation System (XTSVS)* (Section 6.1, Step 7d)
- *Implementation Guidance for FIPS 140-2 and the Cryptographic Module Validation Program* (Section A.9 XTS-AES Key Generation Requirements)

Interface Debug

AXI4-Stream Interfaces

The following is a scenario that you might come across during simulation of the core:

- **s_axis_tready not asserting:** `s_axis_tready` is asserted only when the metadata (key/IV) for the corresponding packet is received by the core as per the metadata handshake protocol. Ensure that the key handshake has successfully taken place for the corresponding packet. `s_axis_tready` can also remain deasserted if the pipeline is completely filled and the `m_axis_tready` is deasserted.

Other Interfaces

The following is a scenario that you might come across on the metadata (key/IV) handshake interface during simulation of the core:

- **key_fetch not asserting:** In case of high-throughput mode of operation, if you see `key_fetch` not asserting after awhile, it might be due to more than one packet inside the pipeline. Ensure that `m_axis_tready` is asserted to flush out one or more packets from the pipeline after which `key_fetch` would be asserted again. This is done to ensure that the core does not accept extra keys and overwrite the existing ones.

Appendix D

Additional Resources and Legal Notices

Xilinx Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see Xilinx Support.

Documentation Navigator and Design Hubs

Xilinx® Documentation Navigator (DocNav) provides access to Xilinx documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the Vivado® IDE, select **Help → Documentation and Tutorials**.
- On Windows, select **Start → All Programs → Xilinx Design Tools → DocNav**.
- At the Linux command prompt, enter `docnav`.

Xilinx Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.
- On the Xilinx website, see the Design Hubs page.

Note: For more information on DocNav, see the Documentation Navigator page on the Xilinx website.

References

These documents provide supplemental material useful with this guide:

1. AES Engine: Federal Information Processing Standards Publication 197 - ADVANCED ENCRYPTION STANDARD (AES) (https://www.nist.gov/publications/advanced-encryption-standard-aes)
2. AES ECB, CFB128: NIST Special Publication 800-38A - Recommendation for Block Cipher Modes of Operation (https://csrc.nist.gov/publications/detail/sp/800-38a/final)
3. NIST Special Publication 800-38E - Recommendation for Block Cipher Modes of Operation: The XTS-AES Mode for Confidentiality on Storage Devices (https://csrc.nist.gov/publications/detail/sp/800-38e/final)
4. https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/details?product=12164
5. https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/details?product=12163
6. *The XTS-AES Validation System (XTSVS)* (Section 6.1, Step 7d)
7. *Implementation Guidance for FIPS 140-2 and the Cryptographic Module Validation Program* (Section A.9 XTS-AES Key Generation Requirements)
8. *Vivado Design Suite: AXI Reference Guide* (UG1037)
9. *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994)
10. *Vivado Design Suite User Guide: Designing with IP* (UG896)
11. *Vivado Design Suite User Guide: Getting Started* (UG910)
12. *Vivado Design Suite User Guide: Logic Simulation* (UG900)

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
04/26/2022 Version 1.1	
Performance	• Added Table 1. • Added Figure 2.
06/03/2020 Version 1.1	
Initial release.	N/A

Please Read: Important Legal Notices

The information disclosed to you hereunder (the "Materials") is provided solely for the selection and use of Xilinx products. To the maximum extent permitted by applicable law: (1) Materials are made available "AS IS" and with all faults, Xilinx hereby DISCLAIMS ALL WARRANTIES AND CONDITIONS, EXPRESS, IMPLIED, OR STATUTORY, INCLUDING BUT NOT LIMITED TO WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, OR FITNESS FOR ANY PARTICULAR PURPOSE; and (2) Xilinx shall not be liable (whether in contract or tort, including negligence, or under any other theory of liability) for any loss or damage of any kind or nature related to, arising under, or in connection with, the Materials (including your use of the Materials), including for any direct, indirect, special, incidental, or consequential loss or damage (including loss of data, profits, goodwill, or any type of loss or damage suffered as a result of any action brought by a third party) even if such damage or loss was reasonably foreseeable or Xilinx had been advised of the possibility of the same. Xilinx assumes no obligation to correct any errors contained in the Materials or to notify you of updates to the Materials or to product specifications. You may not reproduce, modify, distribute, or publicly display the Materials without prior written consent. Certain products are subject to the terms and conditions of Xilinx's limited warranty, please refer to Xilinx's Terms of Sale which can be viewed at https://www.xilinx.com/legal.htm#tos; IP cores may be subject to warranty and support terms contained in a license issued to you by Xilinx. Xilinx products are not designed or intended to be fail-safe or for use in any application requiring fail-safe performance; you assume sole risk and liability for use of Xilinx products in such critical applications, please refer to Xilinx's Terms of Sale which can be viewed at https://www.xilinx.com/legal.htm#tos.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2020-2022 Xilinx, Inc. Xilinx, the Xilinx logo, Alveo, Artix, Kintex, Kria, Spartan, Versal, Vitis, Virtex, Vivado, Zynq, and other designated brands included herein are trademarks of Xilinx in the United States and other countries. All other trademarks are the property of their respective owners.